

# STRATEX Full Validation Report

---

- [STRATEX KEYHORSE TECHNICAL VALIDATION — COMPREHENSIVE INVESTOR DILIGENCE REPORT](#)
  - [TABLE OF CONTENTS](#)
- [PART 1 — REPOSITORY INVENTORY](#)
  - [1. Repositories in scope](#)
  - [2. `TCCross1/stratex-centcom`](#)
  - [3. External applications referenced but not present as code](#)
- [PART 2 — TECHNICAL VALIDATION REPORT](#)
  - [1. Scope limitation \(read first\)](#)
  - [2. Architecture validation](#)
  - [3. Subsystem classification](#)
  - [4. Every mocked / simulated / stubbed / synthetic-data dependency identified](#)
  - [5. What specifically requires real DJI property scans](#)
  - [6. Remaining software work that does NOT depend on field data](#)
  - [7. Build, test, and security evidence \(summary — full detail in Part 4 \(Test and Build Evidence\) below\)](#)
  - [8. Cross-cutting audits](#)
  - [9. Final investor-diligence conclusion](#)
  - [10. Determination](#)
- [PART 3 — FIELD DATA BLOCKERS](#)
  - [1. M4E/M4T imagery ingestion](#)
  - [2. RTK metadata](#)
  - [3. Reconstruction outputs \(general\)](#)
  - [4. Point clouds](#)
  - [5. Meshes](#)
  - [6. Orthomosaics](#)
  - [7. Radiometric R-JPEG thermal data](#)
  - [8. Semantic geometry](#)
  - [9. Property Object Graph object extraction](#)
  - [10. Real-world measurement tolerances](#)
  - [11. Cortex findings/confidence](#)
  - [12. Core quantity/estimate validation](#)
  - [13. Passport truth promotion](#)
  - [14. Report accuracy](#)
  - [15. Pro/Habitat real property synchronization](#)
  - [16. Before/after verification workflows](#)
  - [Summary table](#)
- [PART 4 — TEST AND BUILD EVIDENCE](#)
  - [1. Dependency install](#)
  - [2. Build](#)
  - [3. Unit / domain test suites](#)
  - [4. Lint / type checks](#)
  - [5. Security / static-analysis checks](#)
  - [6. Existing CI-equivalent commands](#)
  - [7. Summary](#)

## STRATEX KEYHORSE TECHNICAL VALIDATION — COMPREHENSIVE INVESTOR DILIGENCE REPORT

---

**Repository audited:** `TCCross1/stratex-centcom` **HEAD commit:** `a69d35247d666f18b6d57038acc1645b7da6b12f` **Audit date (UTC):** 2026-09-11 **Method:** Fresh, evidence-based inspection of the repository itself — no prior status report was assumed correct. Every claim below is backed by direct source citation, direct command execution, or both.

This single file consolidates the four separate audit deliverables into one document: the main technical validation report, the repository inventory, the field-data blocker analysis, and the full test/build evidence log.

---

### TABLE OF CONTENTS

- [1. Repository Inventory](#)
  - [2. Technical Validation Report](#)
  - [3. Field Data Blockers](#)
  - [4. Test and Build Evidence](#)
- 

## PART 1 — REPOSITORY INVENTORY

---

**Audit date (UTC):** 2026-09-11T05:30:34Z **Auditor scope:** Evidence-based inspection of the repository actually present in this sandbox. No prior status report was assumed correct; every claim below was independently re-derived from source, `git`, and command output.

---

## 1. Repositories in scope

Only one Stratex repository was available to this audit environment: `TCCross1/stratex-centcom`.

The directive requests findings “for every Stratex repository,” including references to standalone Stratex Pro and Stratex Habitat applications. This sandbox contains **no clone of a Pro repository, no clone of a Habitat repository, and no clone of any Core/Cortex/reconstruction backend repository**. Their existence, state, and code quality **cannot be verified** from this environment. Everything this audit says about Pro, Habitat, Core, and reconstruction services is derived **only** from how `stratex-centcom` references them (client-side adapter stubs), not from inspecting their actual code.

This is itself a diligence finding: **repository fragmentation cannot be confirmed either way**. If Pro/Habitat/Core exist as separate codebases, they were not provided to this audit and must be separately audited before any investor claim is made about their state.

## 2. `TCCross1/stratex-centcom`

| Field                                      | Value                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Evidence                                                  |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Repository name                            | <code>stratex-centcom</code> (package name: <code>stratex-centcom</code> )                                                                                                                                                                                                                                                                                                                                                                                      | <code>package.json:2</code>                               |
| Active branch                              | <code>copilot/stratex-technical-validation-audit</code>                                                                                                                                                                                                                                                                                                                                                                                                         | <code>git branch --show-current</code>                    |
| Remote                                     | <code>http://localhost:26831/TCCross1/stratex-centcom</code> (sandboxed proxy of <code>github.com/TCCross1/stratex-centcom</code> )                                                                                                                                                                                                                                                                                                                             | <code>git remote -v</code>                                |
| HEAD commit SHA                            | <code>a69d35247d666f18b6d57038acc1645b7da6b12f</code>                                                                                                                                                                                                                                                                                                                                                                                                           | <code>git log -1</code>                                   |
| Prior commit                               | <code>b1e31d01395cf854b593cab4447218cb02c99a19</code>                                                                                                                                                                                                                                                                                                                                                                                                           | <code>git log --oneline</code>                            |
| Total commit count reachable in this clone | 2                                                                                                                                                                                                                                                                                                                                                                                                                                                               | <code>git log --all --oneline</code>                      |
| Shallow clone marker                       | present, boundary at <code>b1e31d0</code>                                                                                                                                                                                                                                                                                                                                                                                                                       | <code>.git/shallow</code>                                 |
| Last meaningful development status         | HEAD commit message is “Update push instructions for the existing TCCross1 repository” — an edit to <code>PUSH.md</code> , not application code. The prior commit <code>b1e31d0</code> (“Add push instructions”) is the actual code-bearing initial commit. <b>This repository was pushed to GitHub as a single squashed snapshot; no incremental development history exists in git.</b> All claimed engineering history is undocumented outside this snapshot. | <code>git log --oneline</code> , <code>PUSH.md</code>     |
| Application/service purpose                | “STRATEX CENTCOM — Command, Control, Intelligence,” described in-repo as “Mission control for residential property intelligence,” the operational command layer that supervises ATC, Evidence, Reality, Cortex, Passport, and the embedded Core engine, and oversees (but does not contain) the standalone Pro and Habitat applications.                                                                                                                        | <code>package.json:5</code> , <code>README.md:1-45</code> |

### Repository structure

```
stratex-centcom/
├─ index.html, package.json, vite.config.js, package-lock.json
├─ PUSH.md           ─ one-time git/GitHub push instructions (not app documentation)
├─ README.md         ─ architecture, honesty constraints, known disconnected providers
├─ public/brand/      ─ one approved logo asset; others explicitly flagged official:false
├─ scripts/run-tests.mjs ─ custom Node test-suite runner (no external test framework)
├─ src/
│  ├─ app/            ─ routing, session bootstrap, viewport hooks, error boundary
│  ├─ components/     ─ navigation chrome, panels, shared domain UI
│  ├─ data/           ─ dashboard-level fixtures (session, alerts, metrics)
│  ├─ design/         ─ design tokens / brand constants
│  ├─ domains/        ─ one directory per canonical domain (18 domains, see below)
│  ├─ pages/          ─ one module per route (7 route groups, 15 page files)
│  └─ utils/          ─ formatting/identifier helpers
```

### Domains present ( `src/domains/*` )

`atc`, `audit`, `awe`, `core`, `cortex`, `evidence`, `habitat`, `measurements`, `mission`, `passport`, `pro`, `property`, `reality`, `reports`, `shared`, `sharing`, `timeline`, `work` — 18 domains, ~6,550 lines of JS across ~60 files ( `wc -l` over `src/domains/*/*.js` ).

### Absent tooling (directly verified)

| Tool/config                             | Present?  | Check performed                                            |
|-----------------------------------------|-----------|------------------------------------------------------------|
| <code>.github/workflows/*</code> (CI)   | Not found | <code>find .github</code> → no such directory              |
| <code>tsconfig*</code> (TypeScript)     | Not found | <code>find . -maxdepth 1 -iname tsconfig*</code> → empty   |
| <code>.eslintrc*</code> / ESLint config | Not found | <code>find . -maxdepth 1 -iname '*.eslint*'</code> → empty |
| Jest/Vitest/Mocha config                | Not found | Test runner is a hand-written <code>scripts/run-</code>    |

|                                         |                    |                                                      |
|-----------------------------------------|--------------------|------------------------------------------------------|
|                                         |                    | tests.mjs that spawnSync s plain Node scripts        |
| .gitignore                              | Present (57 bytes) | ls -la                                               |
| Dockerfile / IaC / deployment manifests | Not found          | find . -iname 'Dockerfile*' -o -iname '*.tf' → empty |

3. External applications referenced but not present as code

| Name                                 | Referenced role in <code>stratex-centcom</code>                                                                                                                                                             | Actual code in this audit?                                    |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Stratex Pro                          | Professional-facing standalone app; consumes Core services and Passport projections via <code>ProAdapter</code> ( <code>src/domains/pro/adapter.js</code> , <code>connected: false</code> )                 | Not in this repo/sandbox                                      |
| Stratex Habitat                      | Homeowner-facing standalone app; consumes Passport projections via <code>HabitatAdapter</code> ( <code>src/domains/habitat/adapter.js</code> , <code>connected: false</code> )                              | Not in this repo/sandbox                                      |
| Core                                 | “the work/execution engine,” described as embedded between Cortex and Passport, represented only by <code>src/domains/core/adapter.js</code> ( <code>connected: false</code> ) and a visibility policy stub | Not in this repo/sandbox — only a client-side stand-in exists |
| Reconstruction/photogrammetry engine | <code>PropertyRealityProcessingProvider</code> , mode <code>FIXTURE</code> , vendor literally recorded as “development fixture – no reconstruction engine connected”                                        | Not present anywhere                                          |

No investor claim should treat Pro, Habitat, Core, or a reconstruction engine as “built” based on this repository — `stratex-centcom` contains only the CENTCOM-side stub interfaces that such systems would eventually satisfy.

PART 2 — TECHNICAL VALIDATION REPORT

**Prepared for:** investor technical diligence **Audit date (UTC):** 2026-09-11T05:30:34Z **Repository audited:** `TCCross1/stratex-centcom` , HEAD `a69d35247d666f18b6d57038acc1645b7da6b12f` , branch `copilot/stratex-technical-validation-audit` **Method:** Direct inspection of source, direct execution of install/build/test commands, no reliance on prior status reports, marketing copy, or README claims without independent re-verification.

This report intentionally does not soften findings. Where a capability is a fixture, it is labeled a fixture. Where a claim in the repository’s own README was checked, the check and its result are shown.

1. Scope limitation (read first)

Only **one** repository was available: `stratex-centcom` . It is a Vite/React single-page application representing the CENTCOM “command layer.” There is no Pro repository, no Habitat repository, no Core/Cortex backend, and no reconstruction-engine repository in this environment. Every statement below about Pro, Habitat, Core, or reconstruction is a statement about **how CENTCOM references them**, not about their own codebases, because those codebases were not provided. See Part 1 (Repository Inventory) above for full detail.

2. Architecture validation

2.1 Claimed pipeline

Capture/DJI input → ATC → Reality Foundry/reconstruction interfaces → Cortex → **Core** → Passport → Comprehensive Property Intelligence Report → 3D Digital Twin + Layers → standalone Stratex Pro and Stratex Habitat.

2.2 What is actually present in code

| Stage                                       | Present in code?                                                                                                                                                                                     | Evidence                                                                                                                                                                                   |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Capture / DJI input                         | Data <b>model</b> only (fixture evidence named <code>DJI_0412.JPG</code> , aircraft IDs <code>AC-M4TD-01/02</code> ); no DJI SDK, no ingestion pipeline                                              | <code>src/domains/evidence/vault-fixtures.js:17-174</code> , <code>src/domains/atc/providers.js:100-128</code> ( <code>AircraftTelemetryProvider</code> , mode <code>DISCONNECTED</code> ) |
| ATC                                         | Real state machine and readiness policy logic; all upstream data providers (weather, airspace, flight, telemetry, sensor) are <code>FIXTURE</code> or <code>DISCONNECTED</code>                      | <code>src/domains/atc/readiness.js</code> , <code>src/domains/atc/policy.js</code> , <code>src/domains/atc/providers.js:14-19</code>                                                       |
| Reality Foundry / reconstruction interfaces | Type registry and provider <b>contract</b> exist; no photogrammetry/LiDAR/CAD engine is connected; provider vendor field literally states “development fixture – no reconstruction engine connected” | <code>src/domains/reality/types.js</code> , <code>src/domains/reality/providers.js:20-24</code>                                                                                            |
| Cortex                                      | Query/read logic over fixture findings/analyses; no                                                                                                                                                  | <code>src/domains/cortex/service.js</code> ,                                                                                                                                               |

|                                            |                                                                                                                                                                                                                            |                                                                                                                               |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
|                                            | model, no inference, no confidence computation beyond static fixture values                                                                                                                                                | src/domains/cortex/fixtures.js                                                                                                |
| Core                                       | Represented <b>only</b> by a client-side adapter returning fixed status strings; <code>connected: false</code> . No execution engine exists in this repository.                                                            | src/domains/core/adapter.js:9-10                                                                                              |
| Passport                                   | Read-only service over a single hard-coded property record ( <code>SXP-004182</code> ); no write path exposed here                                                                                                         | src/domains/passport/service.js ,<br>src/domains/passport/fixtures.js                                                         |
| Comprehensive Property Intelligence Report | Eligibility/manifest <b>decision logic</b> exists (which report modules qualify and why); no renderer, no PDF generation; <code>CoreReportAdapter.connected = false</code> and <code>requestReport()</code> always rejects | src/domains/reports/manifest.js ,<br>src/domains/reports/service.js:40-49                                                     |
| 3D Digital Twin + Layers                   | Type/state vocabulary and layer list defined ( <code>TWIN_LAYERS</code> , 20 layers); “PropertyRealityViewer” component is a stub with no 3D rendering engine wired                                                        | src/domains/reality/types.js:27-31 ; component confirmed as stub (no Three.js/WebGL dependency in <code>package.json</code> ) |
| Stratex Pro (standalone)                   | Referenced only via <code>ProAdapter</code> , <code>connected: false</code> , generates a static mock deep-link URL                                                                                                        | src/domains/pro/adapter.js                                                                                                    |
| Stratex Habitat (standalone)               | Referenced only via <code>HabitatAdapter</code> , <code>connected: false</code> , generates a static mock deep-link URL                                                                                                    | src/domains/habitat/adapter.js                                                                                                |

## 2.3 Is Core embedded inside CENTCOM between Cortex and Passport?

**Confirmed by design intent, not by working software.** `src/domains/core/adapter.js` header comment states: “CENTCOM supervises the engine; it does not run it.” Passport’s fixture record shows a revision committed by `pipeline:core` ( `passport/fixtures.js:15` ), consistent with Core sitting between Cortex and Passport in the data flow. However, **no executable engine exists** — `CoreAdapter` returns static strings ( “Scope computed” , “Complete • 41 surfaces” ) with `connected: false` ( `core/adapter.js:9-34` ). The architectural *position* is correctly modeled; the *engine* is not built.

## 2.4 Are Pro and Habitat standalone product applications?

**Confirmed as intended architecture, not as verified fact.** Both adapters explicitly document that their real applications are separate and “not built inside CENTCOM” ( `pro/adapter.js:1-6` , `habitat/adapter.js:1-4` ). No such applications were available to inspect. Their deep-link URLs ( `https://pro.stratex.app/...` , `https://habitat.stratex.app/...` ) are hard-coded strings in fixture data, not evidence that those domains/services exist or resolve.

## 3. Subsystem classification

Classification legend: **IMPLEMENTED** (real logic, real data path, would work against a live backend) · **IMPLEMENTED / MOCK DATA** (real logic, but all data is fixture) · **PARTIAL** (real but incomplete, e.g. read-only or missing enforcement) · **DESIGNED / NOT IMPLEMENTED** (types/contracts exist, no working logic) · **BLOCKED BY REAL FIELD DATA** (cannot be meaningfully completed without real DJI captures).

| Domain                                     | Classification             | Basis                                                                                                                                                                                                                                                                                       |
|--------------------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ATC                                        | IMPLEMENTED / MOCK DATA    | Real readiness state machine ( <code>readiness.js</code> , 322 lines) and mission-gating policy; all 5 upstream providers are <code>FIXTURE/DISCONNECTED</code>                                                                                                                             |
| Evidence Vault                             | IMPLEMENTED / MOCK DATA    | Real SHA-256 hashing ( <code>integrity.js:38-50</code> ), real immutability enforcement ( <code>storage.js:56-57</code> ), real append-only custody ( <code>vault.js</code> ); storage backend is an in-memory <code>Map</code> , source data is fixture                                    |
| Mission                                    | IMPLEMENTED / MOCK DATA    | Real 15-stage lifecycle state machine with whitelisted transitions ( <code>lifecycle.js</code> ); history is fixture-seeded once at module load ( <code>history.js:111-127</code> ), does not grow with real activity                                                                       |
| Reality / Digital Twin                     | IMPLEMENTED / MOCK DATA    | Real type/version model, real property-scoping and eligibility rules; <code>PropertyRealityProcessingProvider.mode = FIXTURE</code> ; every produced descriptor is explicitly <code>isSimulated: true</code> with null point/vertex/face counts ( <code>reality/providers.js:36-54</code> ) |
| Cortex                                     | IMPLEMENTED / MOCK DATA    | Real query/filter logic over analyses/findings; all analyses, findings, confidence values are static fixtures ( <code>cortex/fixtures.js</code> ); no inference engine exists                                                                                                               |
| Core                                       | DESIGNED / NOT IMPLEMENTED | Only a status-returning adapter stub; <code>connected: false</code> ; no measurement/takeoff/estimating engine present                                                                                                                                                                      |
| Passport                                   | PARTIAL                    | Read-only over one fixture property record; no write path, no conflict resolution, no multi-property support demonstrated                                                                                                                                                                   |
| Comprehensive Property Intelligence Report | DESIGNED / NOT IMPLEMENTED | Manifest/eligibility decision logic is real; no rendering/PDF generation exists; <code>CoreReportAdapter.connected = false</code> , <code>requestReport()</code> always rejects ( <code>reports/service.js:47-48</code> )                                                                   |
| Pro (standalone)                           | DESIGNED / NOT IMPLEMENTED | Client-side stub adapter only; no separate codebase                                                                                                                                                                                                                                         |

|                                                   |                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                   |                                                     | provided                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Habitat (standalone)                              | DESIGNED / NOT IMPLEMENTED                          | Client-side stub adapter only; no separate codebase provided                                                                                                                                                                                                                                                                                                                                                                                    |
| Property / directory                              | IMPLEMENTED / MOCK DATA                             | Real aggregation/summary logic ( <code>property/service.js</code> ); all rows are fixture                                                                                                                                                                                                                                                                                                                                                       |
| Measurements                                      | IMPLEMENTED / MOCK DATA                             | Real truth-class accounting; fixture measurement values                                                                                                                                                                                                                                                                                                                                                                                         |
| AWE (Automated Weather/Energy — as named in code) | IMPLEMENTED / MOCK DATA                             | Real query logic; scoring explicitly fixture ( <code>aweByProperty</code> fixtures)                                                                                                                                                                                                                                                                                                                                                             |
| Work (projects/repairs/maintenance)               | IMPLEMENTED / MOCK DATA                             | Real grouping/filtering logic over fixture project/repair/maintenance rows                                                                                                                                                                                                                                                                                                                                                                      |
| Sharing                                           | IMPLEMENTED / MOCK DATA                             | Real scope/grant logic over fixture grants                                                                                                                                                                                                                                                                                                                                                                                                      |
| Audit                                             | PARTIAL / MOCK DATA                                 | Read-only listing; no independent immutability enforcement observed in this module (append-only is a documented convention, not a mechanism inside <code>audit/service.js</code> )                                                                                                                                                                                                                                                              |
| Timeline                                          | PARTIAL / MOCK DATA — <b>property isolation gap</b> | <code>TimelineService.listByProperty(propertyId)</code> <b>ignores its <code>propertyId</code> parameter and returns all events regardless of property</b> ( <code>src/domains/timeline/service.js:8</code> ). This is a real, present-day bug, not a field-data blocker, and it contradicts the “no cross-property bleed” architectural rule enforced elsewhere.                                                                               |
| Shared (classification, RBAC, transport, states)  | IMPLEMENTED                                         | Pure, fixture-free vocabulary/policy definitions ( <code>TRUTH_CLASS</code> , <code>REVIEW_STATE</code> , <code>ROLE_SCOPES</code> ); these are real and load-bearing, but <b>RBAC is defined and displayed, not enforced</b> — <code>ROLE_SCOPES</code> is referenced in exactly one place in the whole app, a read-only admin panel table ( <code>components/panels/DownstreamPanels.jsx:170</code> ), not in any route guard or service call |
| Authentication / session                          | DESIGNED / NOT IMPLEMENTED                          | <code>centcomApi.getSession()</code> resolves a hard-coded fixture object ( <code>src/domains/index.js:41-42</code> ); no login flow, no identity provider, no token validation anywhere in the codebase                                                                                                                                                                                                                                        |

#### 4. Every mocked / simulated / stubbed / synthetic-data dependency identified

Grep-verified marker classes and representative locations (not exhaustive line lists — full detail in Part 4 (Test and Build Evidence) below):

- **sourceMode: "FIXTURE"** — `reality/providers.js:48` , and stamped onto every ATC provider payload via `stamp()` ( `atc/providers.js:26-30` )
- **isSimulated: true** — every reconstruction artifact descriptor ( `reality/providers.js:49` )
- **PROVIDER\_MODE.DISCONNECTED** — `AirspaceProvider` ( `atc/providers.js:54` ), `FlightProvider` , `AircraftTelemetryProvider` ( `atc/providers.js:100` )
- **connected: false** — `CoreAdapter` ( `core/adapters.js:10` ), `ProAdapter` ( `pro/adapters.js:10` ), `HabitatAdapter` ( `habitat/adapters.js:10` ), `CoreReportAdapter` ( `reports/service.js:41` )
- **STORAGE\_MODE.IN\_MEMORY\_STORAGE** — `evidence/storage.js:49` ; storage is a `JS Map` , not a database or object store
- **HASH\_STATE.NOT\_COMPUTED\_FIXTURE** — `evidence/integrity.js:16,77-83` — fixture evidence never gets a fabricated hash
- **"vendor": "development fixture – no reconstruction engine connected"** — `reality/providers.js:23`
- **Deterministic DJI-named fixtures** — `DJI_0412.JPG` , `DJI_0412_R.JPG` , `AC-M4TD-01/02` throughout `evidence/vault-fixtures.js` and `evidence/fixtures.js` — filenames only, no real image bytes, no EXIF/RTK metadata parsing
- **Working (non-final) product names** — the three digital twin types are `TWIN_TYPE_A/B/C` with `permanentNameLocked: false` , enforced by a test ( `reality/types.js:16` , README “Naming note”)
- **Non-final brand assets** — only `stratex-centcom-logo.png` is `official: true` ; all other brand assets are `official: false` (README “Branding”)
- **Heuristic, unvalidated scoring** — the ATC thermal-suitability prediction explicitly disclaims itself: *“Predicted suitability from a development heuristic. Not a calibrated or scientifically validated model”* ( `atc/policy.js:82-83` )
- **Hard-coded session/identity** — `centcomApi.getSession()` ( `src/domains/index.js:41-42` )

#### 5. What specifically requires real DJI property scans

| Requirement in directive                                    | Current state                                                                                                                                                                                                                                       | What real capture would change                                                                                    |
|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| M4E/M4T imagery                                             | Fixture filenames only ( <code>DJI_0412.JPG</code> ); no real image files, no image processing                                                                                                                                                      | Would require an actual ingestion path (upload/API), not currently implemented anywhere                           |
| RTK metadata                                                | ATC readiness references <code>GPS_RTK</code> category and an <code>rtkState</code> field, but values are canned strings ( <code>"READY"</code> / <code>"UNAVAILABLE"</code> ) in fixtures                                                          | Real telemetry would need <code>AircraftTelemetryProvider</code> connected (currently <code>DISCONNECTED</code> ) |
| Reconstruction outputs (point clouds, meshes, orthomosaics) | <code>PropertyRealityProcessingProvider</code> returns descriptors with <code>pointCount</code> , <code>vertexCount</code> , <code>faceCount</code> , <code>resolution</code> <b>all explicitly null</b> — the code refuses to invent these numbers | Requires an actual photogrammetry/LiDAR engine; none exists in any provided repository                            |
| Radiometric R-JPEG thermal data                             | Evidence model defines <code>THERMAL_RJPEG</code> as an artifact type; fixture assets reference thermal filenames and a “Thermal 640x512” sensor label, but                                                                                         | Requires real R-JPEG files and a radiometric parser, neither present                                              |

|                                                             |                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                  |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                             | no radiometric decoding logic exists                                                                                                                                                                           |                                                                                                                                                                                                                                                                  |
| Semantic geometry / Property Object Graph object extraction | Not implemented; no object-graph extraction code found anywhere in <code>src/domains/reality</code> or <code>src/domains/cortex</code>                                                                         | This is a from-scratch build, and its accuracy cannot be validated without real reconstructed geometry to extract from                                                                                                                                           |
| Real-world measurement tolerances                           | <code>measurements</code> domain enforces a <code>TRUTH_CLASS</code> distinction (MEASURED vs DERIVED vs PROBABLE) but has no real measurement source to validate tolerance against; all values are fixture    | Tolerance/accuracy claims are currently unverifiable and unverified — none should be made                                                                                                                                                                        |
| Cortex findings/confidence                                  | Confidence values exist only as static fixture numbers on <code>PROBABLE</code> findings; no model producing them                                                                                              | Cannot be validated as “real” confidence until Cortex runs against real evidence                                                                                                                                                                                 |
| Core quantity/estimate validation                           | <code>CoreAdapter</code> returns static strings like <code>"Complete • 41 surfaces"</code> ; no computation occurs                                                                                             | Entirely blocked — Core does not exist as an engine in this codebase                                                                                                                                                                                             |
| Passport truth promotion                                    | Revision history is fixture; the write path from Cortex/Core into Passport is documented as a rule, not demonstrated as working code path with real inputs                                                     | Needs real upstream findings to promote; current revisions are pre-baked                                                                                                                                                                                         |
| Report accuracy                                             | No renderer exists; <code>requestReport()</code> always rejects                                                                                                                                                | N/A until Core report generation exists; nothing to validate for accuracy yet                                                                                                                                                                                    |
| Pro/Habitat real property synchronization                   | Both adapters return static mock sync status ( <code>"synced"</code> , <code>syncFailures: 0</code> ) with <code>connected: false</code>                                                                       | Cannot be assessed at all without the actual Pro/Habitat codebases and a live property                                                                                                                                                                           |
| Before/after verification workflows                         | <code>work/fixtures.js</code> references “completion evidence” and Passport revisions tied to repair completion, but this is fixture data illustrating the intended shape, not a working verification pipeline | Requires real before/after captures and a real comparison engine ( <code>ComparisonProvider</code> , currently <code>mode: FIXTURE</code> , <code>isLive: () =&gt; false</code> , <code>evidence/...</code> “no geometric difference computation is performed” ) |

See Part 3 (Field Data Blockers) below for the full blocker-by-blocker breakdown.

## 6. Remaining software work that does NOT depend on field data

These items are conventional engineering work that can and should be completed **before** or **independent of** obtaining real DJI scans:

1. **Fix the timeline property-isolation bug** — `TimelineService.listByProperty()` ignores its parameter and returns all events for all properties ( `src/domains/timeline/service.js:8` ). This directly violates the “no cross-property bleed” invariant enforced everywhere else and should be fixed immediately; it is a software bug, not a data-availability issue.
2. **Enforce RBAC, don’t just declare it.** `ROLE_SCOPES / can()` exist ( `shared/rbac.js` ) but are referenced in exactly one read-only UI panel and nowhere else — no route guard, no service-call authorization check anywhere in `src/app` or `src/domains` .
3. **Implement real authentication/session management.** `getSession()` is a hard-coded fixture ( `domains/index.js:41-42` ); there is no login, token, or identity-provider integration.
4. **Add TypeScript or at least a static type-checking layer.** No `tsconfig.json` exists; all ~60 source files are untyped `.js / .jsx` .
5. **Add a linter.** No `.eslintrc*` exists; code style is convention-only.
6. **Add CI/CD.** No `.github/workflows/*` exists; there is no automated gate on tests, build, or lint for any pull request.
7. **Persist evidence storage to a real database/object store.** `EvidenceStorageProvider` is a JS `Map` that vanishes on process restart ( `evidence/storage.js:44-49` ). Building the real storage contract implementation is pure engineering work — the *interface* is already defined and does not require field data to implement.
8. **Build the Core report renderer and manifest execution.** `ReportManifestService` decision logic is real; the actual rendering/PDF generation step ( `CoreReportAdapter` ) does not exist and is conventional software engineering, not something requiring field data per se (though its content will need real evidence to populate later).
9. **Wire provider health monitoring into the UI.** Every provider defines a `health()` method (e.g. `atc/providers.js` health endpoints, `reality/realityProviderHealth()` ), but nothing polls or surfaces them on an operational dashboard.
10. **Expand fixture data to multiple properties for realistic multi-tenant testing**, and add tests that exercise Passport’s write/conflict path (currently read-only in this codebase).
11. **Address the two npm audit vulnerabilities** (see §7) — moderate/high severity in `esbuild / vite` dev dependencies, fixable via a version bump, unrelated to field data.
12. **Resolve the architectural ambiguity around Pro/Habitat/Core repositories.** If they exist elsewhere, produce them for audit; if they don’t yet exist, that is itself a major scope item independent of field-data availability.

None of the above requires a drone flight to complete.

## 7. Build, test, and security evidence (summary — full detail in Part 4 (Test and Build Evidence) below)

- `npm install` — succeeded, 63 packages added, 2 vulnerabilities reported (1 moderate, 1 high, both in `esbuild / vite` dev tooling).
- `npm test` ( `node scripts/run-tests.mjs` ) — **all 5 suites pass**: Mission 65/65, ATC 79/79, Evidence Vault 107/107, Property Reality 144/144, Property Isolation full sweep pass. Total 395 discrete `check()` assertions across the four counted suites, independently re-run and confirmed, matching the README’s claim.
- `npm run build` ( `vite build` ) — succeeded, produced `dist/index.html` and one 666.82 kB JS bundle (171.61 kB gzip), with a Vite warning about chunk size (no code-splitting configured).
- No lint command exists to run (no ESLint config).
- No TypeScript command exists to run (no `tsconfig.json` ).
- No CI workflow exists to reference — the audit ran these commands directly and is the first independent execution/verification of this claim on record in this environment.

## 8. Cross-cutting audits

| Area                                 | Finding                                                                                                                                                                                                                                 | Evidence                                                                           |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Evidence lineage                     | Property → mission → capture → evidence → (analysis/finding) → Passport revision chain is fully modeled with ID references at every hop                                                                                                 | evidence/vault-fixtures.js , cortex/fixtures.js:16-71 , passport/fixtures.js:14-17 |
| Audit/version history                | audit/service.js is read-only listing; Passport revisions are append-style ( r12 → r13 → r14 ) but no separate audit-service enforcement mechanism was found beyond convention                                                          | audit/service.js , passport/fixtures.js                                            |
| Property isolation                   | Enforced and tested for 15 of 16 scoped record types via isolation.test.mjs ; <b>Timeline is the one confirmed exception</b> (§3, §6)                                                                                                   | src/domains/isolation.test.mjs:30-40 , timeline/service.js:8                       |
| Security controls                    | RBAC model defined, not enforced (§6); no authentication; no server-side anything (this is a client-only SPA with an in-memory data layer)                                                                                              | shared/rbac.js , domains/index.js:41-42                                            |
| Data contracts                       | Every provider/adaptor declares an explicit mode/interface ( PROVIDER_MODE , STORAGE_MODE ) — a genuinely good practice that makes swapping in real backends traceable                                                                  | atc/providers.js:14-19 , evidence/storage.js:14-20                                 |
| API boundaries                       | No real API layer exists; serve() ( shared/transport.js ) is an in-process Promise / setTimeout wrapper simulating network latency (260ms), not an HTTP client                                                                          | shared/transport.js:9-22                                                           |
| Failure handling                     | serve() supports a fail flag and providers throw explicit errors when disconnected/live-but-unimplemented ( reality/providers.js:62 ); UI has an error boundary ( Boundary component in App.jsx )                                       | shared/transport.js , reality/providers.js:56-63                                   |
| Persistence                          | None beyond in-memory Map /arrays reset on process restart; no database anywhere in the stack                                                                                                                                           | evidence/storage.js:44-49 , all */fixtures.js                                      |
| Report generation                    | Decision/eligibility logic real; no renderer; no report ever produced                                                                                                                                                                   | reports/manifest.js , reports/service.js:40-49                                     |
| Deployment/configuration assumptions | No environment-variable-driven config found; no Dockerfile, no infra-as-code, vite.config.js is default/minimal                                                                                                                         | vite.config.js                                                                     |
| Third-party dependency risk          | Only 2 runtime deps ( react , react-dom ) and 2 dev deps ( vite , @vitejs/plugin-react ); npm audit shows esbuild/vite dev-only vulnerabilities (moderate/high), fixable with a breaking upgrade path ( npm audit fix --force → Vite 8) | package.json:11-12 , npm audit output                                              |

## 9. Final investor-diligence conclusion

**A. What is demonstrably built today?** A single-repository React/Vite front-end ( stratex-centcom ) with a genuinely well-structured domain layer: real state machines (mission lifecycle, ATC readiness), real cryptographic evidence hashing (SHA-256 via WebCrypto), real property-scoping/isolation enforcement (verified by a passing 395-assertion, 5-suite test run), real truth-classification discipline (MEASURED/DERIVED/PROBABLE), and a consistent, explicit convention for labeling every mock, fixture, and disconnected provider. The build succeeds and all existing tests pass.

**B. What works using synthetic/mock data?** Everything that touches an external system or produces a data value: ATC's weather/airspace/flight/telemetry/sensor providers, Evidence's storage and DJI-style source ingestion, Reality's reconstruction pipeline (point clouds/meshes/orthomosaics — all descriptor-only with null counts), Cortex's findings/confidence, Passport's single fixture property record, and Core/Pro/Habitat's status adapters. This is the large majority of what an investor would see as "the product working."

**C. What remains conventional software engineering work (independent of field data)?** Authentication, RBAC enforcement, a real persistence layer, TypeScript/lint/CI tooling, the Core report renderer, provider health monitoring, the timeline property-isolation bug fix, and dependency-vulnerability remediation. This is real, scoped, doable work that does not require a drone flight — see §6.

**D. What specifically cannot be responsibly completed or validated without real property scans?** Any claim of measurement accuracy or tolerance, any claim about reconstruction fidelity (point cloud density, mesh quality, orthomosaic resolution), radiometric thermal interpretation, semantic/object-graph extraction from real geometry, Cortex confidence calibration against real findings, Core quantity/estimate correctness, Passport truth-promotion behavior under real conflicting inputs, and any before/after verification claim. See Part 3 (Field Data Blockers) above.

**E. Is it technically reasonable to state: "Stratex has reached the pre-field-validation boundary, where the next meaningful development stage requires real-world property capture data"?**

**Partially, with material caveats — not as an unqualified statement.** The domain logic, state machines, and honesty-by-construction patterns in stratex-centcom are genuinely at a point where real capture data would meaningfully advance the reconstruction/Cortex/Core/report chain. However, that statement **cannot be made about the whole platform**, because: - Authentication, RBAC enforcement, persistence, CI/lint/type-checking, and the Core report renderer are **not field-data-blocked** — they are unstated or partially-started conventional engineering work that should happen regardless of when field data arrives. - Core, Pro, and Habitat exist in this audit only as client-side stubs; their actual state is unknown and unverifiable from what was provided. - A real, present-day bug exists (timeline cross-property isolation) that has nothing to do with field data and should be fixed first.

The correct, non-inflated framing for investors is: **"The CENTCOM domain-modeling and evidence-handling layer has reached a point where its reconstruction, Cortex, and Core-facing interfaces are ready to be exercised by real capture data — but the platform as a whole has not reached a field-validation boundary, because material non-field-dependent engineering (auth, persistence, CI, Core/Pro/Habitat implementation) remains open and unverified."**

## 10. Determination

### Ready to enter funded field-validation phase: PASS WITH CONDITIONS

Conditions: 1. Fix the confirmed timeline property-isolation defect before any real multi-property field data is ingested. 2. Implement real authentication and enforce the already-defined RBAC before any real property data (which is sensitive) is loaded into this system. 3. Replace in-memory storage with real persistence before treating any field-validation run as durable/auditable. 4. Produce (or provide for audit) the actual Pro, Habitat, and Core repositories — their current absence means no investor claim about “the full platform” can be verified. 5. Do not present any reconstruction, measurement, or confidence output as accurate until it has been validated against real captures with documented tolerance testing.

## PART 3 — FIELD DATA BLOCKERS

**Purpose:** Enumerate, with code citations, exactly which capabilities in `stratex-centcom` cannot be completed or validated without real DJI property scans — as distinct from work that is merely unfinished software engineering (see the companion technical report §6 for that list).

A “blocker” here means: the code has a defined seam/interface for the real capability, but the only way to prove that seam works correctly is to feed it real capture data. Fixture data can exercise the *shape* of these systems but can never validate their *correctness*.

### 1. M4E/M4T imagery ingestion

- **Current state:** Fixture evidence assets carry filenames modeled on real DJI output naming ( `DJI_0412.JPG` , `DJI_0088.JPG` ) and aircraft IDs `AC-M4TD-01` / `AC-M4TD-02` ( `src/domains/evidence/vault-fixtures.js:17-174` , `src/domains/evidence/fixtures.js:30-91` ). No actual image bytes exist; `byteSize` fields are fixture numbers (e.g. `byteSize: 11200000` ), not measured from real files.
- **Why field data is required:** There is no way to validate that an ingestion path correctly parses real M4E/M4T output (file structure, EXIF, sensor tags) without real files from the aircraft. This is not a logic gap that can be closed with more fixture design — it requires actual captured imagery.

### 2. RTK metadata

- **Current state:** ATC readiness checks a `GPS_RTK` category ( `src/pages/atc/AtcMissionDetailPage.jsx:233` ) and displays an `rtkState` field per capture frame ( `AtcMissionDetailPage.jsx:443` ). `AircraftTelemetryProvider` is `DISCONNECTED` ( `src/domains/atc/providers.js:100-128` ); all RTK values seen in fixtures are canned strings such as `"READY"` .
- **Why field data is required:** RTK fix quality, latency, and correction-stream behavior can only be validated against a live GPS/RTK telemetry stream from an actual aircraft in the field. No amount of fixture data proves the telemetry provider contract is correct against real hardware.

### 3. Reconstruction outputs (general)

- **Current state:** `PropertyRealityProcessingProvider.mode = FIXTURE` ( `src/domains/reality/providers.js:22` ). `produceDescriptor()` explicitly sets `pointCount: null` , `vertexCount: null` , `faceCount: null` , `resolution: null` and stamps `isSimulated: true` with the notice `"SIMULATED DERIVED ARTIFACT — no photogrammetry, LiDAR or CAD processing was performed."` ( `reality/providers.js:36-53` ).
- **Why field data is required:** There is no reconstruction engine in any repository provided to this audit. Building and validating one requires real overlapping imagery/LiDAR from real property flights — this is a hard blocker, not a software-only gap.

### 4. Point clouds

- **Current state:** `SPATIAL_ARTIFACT_TYPE.POINT_CLOUD` is a defined enum value ( `reality/types.js:19` ); format is recorded as `"LAS/LAZ (descriptor only)"` ( `reality/providers.js:78` ); `pointCloudSource` can be `POINT_CLOUD_SOURCE.FIXTURE_POINT_CLOUD` only.
- **Why field data is required:** Point density, noise characteristics, and registration accuracy are properties of a real capture and real processing run; cannot be fabricated or approximated meaningfully with fixtures.

### 5. Meshes

- **Current state:** `MESH` / `TEXTURED_MESH` artifact types defined, format recorded as `"OBJ (descriptor only)"` / `"glTF (descriptor only)"` ( `reality/providers.js:79-80` ). No mesh geometry is ever generated.
- **Why field data is required:** Mesh fidelity (watertightness, topology correctness, texture alignment) is inseparable from the quality of the source imagery/point cloud it's built from.

### 6. Orthomosaics

- **Current state:** `ORTHOMOSAIC` artifact type defined, format `"GeoTIFF (descriptor only)"` ( `reality/providers.js:81` ). No orthomosaic is ever stitched.
- **Why field data is required:** Orthomosaic geometric accuracy depends on real overlapping aerial imagery and ground control — nothing fixture-based can validate this.

### 7. Radiometric R-JPEG thermal data

- **Current state:** `THERMAL_RJPEG` is a defined artifact/origin type ( `src/domains/evidence/model.js:30` ); fixture thermal assets (e.g. `DJI_0412_R.JPEG` , sensor `"Thermal 640x512"` ) exist as filenames/metadata only ( `evidence/fixtures.js:37-43` ). No radiometric decoding (temperature-per-pixel extraction) logic exists anywhere in the codebase.
- **Why field data is required:** Radiometric calibration, environmental compensation (ambient temp, emissivity, reflected temp), and interpretation of a real R-JPEG payload can only be built and validated against real DJI thermal captures.

## 8. Semantic geometry

- **Current state:** Not implemented. No code in `src/domains/reality` or `src/domains/cortex` performs semantic labeling of geometry (e.g. identifying “this mesh region is a roof plane,” “this is a window”).
- **Why field data is required:** This is a from-scratch capability that depends on having real reconstructed geometry to label in the first place — it is blocked transitively by reconstruction (§3) and cannot be prototyped meaningfully on synthetic geometry alone if the goal is real-world accuracy.

## 9. Property Object Graph object extraction

- **Current state:** No object-graph extraction logic found in the codebase. `TWIN_LAYERS` ( `reality/types.js:27-31` ) defines a *vocabulary* of 20 layers (roof, exterior envelope, windows, doors, HVAC, electrical, plumbing, foundation, drainage, etc.) but there is no code that extracts or populates these layers from geometry.
- **Why field data is required:** Object extraction accuracy is only meaningful when measured against real structures; fixture “layers” are just labeled empty categories today.

## 10. Real-world measurement tolerances

- **Current state:** `measurements` domain enforces the `TRUTH_CLASS` distinction ( `MEASURED` vs `DERIVED` vs `PROBABLE` ) as a matter of policy ( `shared/classification.js` ), and the README states “Photogrammetric dimensions are DERIVED, never MEASURED” as a design rule. No actual tolerance/accuracy figures exist anywhere, because no real measurement has ever been taken.
- **Why field data is required:** Tolerance can only be established by comparing derived measurements against ground truth from real properties. Any tolerance number quoted today would be fabricated.

## 11. Cortex findings/confidence

- **Current state:** `cortex/fixtures.js` contains static, hand-authored findings with fixed confidence values. `CortexService` ( `cortex/service.js` ) only filters/queries this static data — there is no inference engine.
- **Why field data is required:** Confidence calibration (e.g., “a finding marked 0.82 confidence is correct 82% of the time”) is an empirical claim that requires a real corpus of evidence and ground-truth outcomes. None exists.

## 12. Core quantity/estimate validation

- **Current state:** `CoreAdapter.getPropertyWorkState()` returns hard-coded strings like “Complete • 41 surfaces” and “Complete • 3 assemblies” ( `core/adapter.js:24-34` ). No quantity computation, takeoff, or estimating logic exists.
- **Why field data is required:** Blocked twice over — first because Core itself does not exist as an engine in any provided repository, and second because even a built engine’s quantity/estimate outputs can only be validated against real measured properties.

## 13. Passport truth promotion

- **Current state:** `passportRecord.revisions` is a static, pre-written array showing what a promotion history *would* look like ( `passport/fixtures.js:8-17` ). `PassportService` is read-only ( `passport/service.js:9-12` ) — there is no code path in this repository that actually promotes a Cortex finding or Core output into a new Passport revision.
- **Why field data is required:** The promotion *rule* (only Cortex/Core write through controlled pathways, professionals never write directly) is documented but unexercised. Validating that it behaves correctly under real conflicting or ambiguous findings requires real-world evidence streams, not just more fixture rows.

## 14. Report accuracy

- **Current state:** `ReportManifestService / buildManifest()` correctly decides which report modules are eligible given available capability flags ( `reports/manifest.js:72-89` ) — this logic is real and testable independent of field data. However, `CoreReportAdapter.requestReport()` unconditionally rejects ( `reports/service.js:47-48` ), so no report is ever actually generated, and no report *content* exists to assess for accuracy.
- **Why field data is required:** Once a renderer exists, report accuracy (are the stated measurements, findings, and thermal readings correct) can only be assessed against a real property with independently verified ground truth.

## 15. Pro/Habitat real property synchronization

- **Current state:** Both `ProAdapter.getOversight()` and `HabitatAdapter.getOversight()` return static mock sync states ( `syncState: “synced”, syncFailures: 0` ) with `connected: false` ( `pro/adapter.js:9-25` , `habitat/adapter.js:9-25` ).
- **Why field data is required:** Real synchronization correctness (data arriving in Pro/Habitat matching the canonical Passport record, latency, conflict handling) can only be verified once real Pro/Habitat applications exist and are fed real property data — currently neither exists as inspectable code, and no real property flows through the system at all.

## 16. Before/after verification workflows

- **Current state:** `work/fixtures.js` models “completion evidence” tied to repairs and Passport revisions, illustrating the intended shape of a before/after comparison. The actual comparison engine, `ComparisonProvider` ( `reality/providers.js:90-100` ), is `mode: FIXTURE`, `isLive: () => false`, and its own health description states plainly: “*Fixture comparison — no geometric difference computation is performed.*”
- **Why field data is required:** Before/after verification is fundamentally a geometric-diffing problem between two real captures of the same property at different times. It cannot be validated — or arguably even meaningfully built — without real repeated captures of real properties.

## Summary table

| # | Item | Blocking reason | Primary citation |
|---|------|-----------------|------------------|
|---|------|-----------------|------------------|

|    |                                   |                                                              |                                          |
|----|-----------------------------------|--------------------------------------------------------------|------------------------------------------|
| 1  | M4E/M4T imagery                   | No real files exist to ingest/parse                          | evidence/vault-fixtures.js:17-174        |
| 2  | RTK metadata                      | Telemetry provider disconnected; no real GPS stream          | atc/providers.js:100-128                 |
| 3  | Reconstruction outputs            | No engine exists; needs real overlapping capture             | reality/providers.js:20-53               |
| 4  | Point clouds                      | Density/accuracy unverifiable without real scans             | reality/providers.js:78                  |
| 5  | Meshes                            | Fidelity tied to real source data quality                    | reality/providers.js:79-80               |
| 6  | Orthomosaics                      | Geometric accuracy needs real aerial imagery                 | reality/providers.js:81                  |
| 7  | R-JPEG thermal                    | No radiometric decoder; needs real thermal files             | evidence/model.js:30                     |
| 8  | Semantic geometry                 | No labeling logic; depends on real geometry                  | n/a (not implemented)                    |
| 9  | Property Object Graph extraction  | No extraction logic; depends on real geometry                | reality/types.js:27-31                   |
| 10 | Measurement tolerances            | No ground truth exists to establish tolerance                | shared/classification.js                 |
| 11 | Cortex confidence                 | No inference engine; confidence is fixture                   | cortex/fixtures.js                       |
| 12 | Core quantity/estimate validation | No engine; needs real measured properties                    | core/adapters.js:24-34                   |
| 13 | Passport truth promotion          | Write path unexercised; needs real conflicting evidence      | passport/service.js:9-12                 |
| 14 | Report accuracy                   | No renderer; content accuracy needs real ground truth        | reports/service.js:47-48                 |
| 15 | Pro/Habitat sync                  | Neither app exists as code; no real property flows           | pro/adapters.js ,<br>habitat/adapters.js |
| 16 | Before/after verification         | Comparison engine is fixture-only, admittedly non-functional | reality/providers.js:90-100              |

## PART 4 — TEST AND BUILD EVIDENCE

**Repository:** TCCross1/stratex-centcom **Commit audited:** a69d35247d666f18b6d57038acc1645b7da6b12f **Environment:** Node v22.23.2 , sandboxed Linux container, fresh npm install (no cached prior state) **Date (UTC):** 2026-09-11

All commands below were executed directly in this audit session. Output is reproduced exactly (trimmed only where noted).

### 1. Dependency install

```
$ npm install
added 63 packages, and audited 64 packages in 9s

7 packages are looking for funding
  run `npm fund` for details

2 vulnerabilities (1 moderate, 1 high)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

**Result: PASS** (installs cleanly).

### 2. Build

```
$ npm run build

> stratex-centcom@0.1.0 build
> vite build

vite v5.4.21 building for production...
transforming...
✓ 117 modules transformed.
rendering chunks...
```

```
computing gzip size...
dist/index.html          0.46 kB | gzip:  0.31 kB
dist/assets/index-CP3H9iDm.js 666.82 kB | gzip: 171.61 kB

(!) Some chunks are larger than 500 kB after minification. Consider:
- Using dynamic import() to code-split the application
- Use build.rollupOptions.output.manualChunks to improve chunking: https://rollupjs.org/configuration-options/#output-manualchunks
- Adjust chunk size limit for this warning via build.chunkSizeWarningLimit.
✓ built in 1.41s
```

**Result: PASS**, with one non-fatal advisory warning (bundle size / no code-splitting). This is a legitimate engineering observation: the entire application ships as a single ~667 kB JS chunk.

### 3. Unit / domain test suites

Command: `npm test` → `node scripts/run-tests.mjs`, which `spawnSync`s five independent Node scripts and aggregates their `RESULT` lines. There is no separate “integration test” suite distinct from these — the repository does not distinguish unit vs. integration; all five are domain-level behavioral suites run against in-process fixture data.

```
$ npm test

> stratex-centcom@0.1.0 test
> node scripts/run-tests.mjs

PASS  Mission          RESULT: ALL 65 TESTS PASS
PASS  ATC               RESULT: ALL 79 TESTS PASS
PASS  Evidence Vault    RESULT: ALL 107 TESTS PASS
PASS  Property Reality   RESULT: ALL 144 TESTS PASS
PASS  Property Isolation RESULT: ALL TESTS PASS

ALL SUITES PASS
```

Exact pass/fail counts, independently re-derived (not taken from the README) by running each suite file directly with `node <file>` and reading its terminal `RESULT` line:

| Suite                          | File                                                | Assertions                                                                                       | Result             |
|--------------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------|
| Mission                        | <code>src/domains/mission/mission.test.mjs</code>   | 65                                                                                               | ALL 65 TESTS PASS  |
| ATC                            | <code>src/domains/atc/atc.test.mjs</code>           | 79                                                                                               | ALL 79 TESTS PASS  |
| Evidence Vault                 | <code>src/domains/evidence/evidence.test.mjs</code> | 107                                                                                              | ALL 107 TESTS PASS |
| Property Reality               | <code>src/domains/reality/reality.test.mjs</code>   | 144                                                                                              | ALL 144 TESTS PASS |
| Property Isolation & Integrity | <code>src/domains/isolation.test.mjs</code>         | Full sweep across 16 relationship classes (no per-assertion count printed; see full output §3.1) | ALL TESTS PASS     |

**Total counted assertions across the four numbered suites: 395** (65+79+107+144), matching the figure previously stated in the repository’s own `README.md`. This is one of the few README claims in this repository that independently re-checks out exactly.

**Result: PASS.** 0 failures observed across all 5 suites in this run.

#### 3.1 Full output — Property Isolation & Integrity suite (representative excerpt, tail)

```
ok Finding → Evidence – every evidence resolves to the same property
ok Finding → Analysis – every analysis resolves to the same property
ok Finding → Mission – every mission resolves to the same property
ok Evidence → Mission – every mission resolves to the same property
ok Analysis → Mission – every mission resolves to the same property
ok Twin → Source Mission – every mission resolves to the same property
ok Repair → Project – every project resolves to the same property
ok Repair → Evidence – every evidence resolves to the same property
ok Project → Findings – every finding resolves to the same property
ok Maintenance → Finding – every finding resolves to the same property
ok Document → Project – every project resolves to the same property
ok Gold → Finding – every finding resolves to the same property
ok Prediction → Finding – every finding resolves to the same property

=== 4. TRUTH DISCIPLINE ===
ok no MEASURED or DERIVED record carries a confidence score
ok every PROBABLE record states its confidence

RESULT: ALL TESTS PASS
```

**Caveat:** this test explicitly checks only relationships **within the domain’s own fixture data** (`src/domains/isolation.test.mjs:9-17` imports fixtures directly). It does **not** cover `TimelineService.listByProperty()`, which is implemented in `src/domains/timeline/service.js` and does not filter by property at all (`listByProperty: (propertyId) => serve(() => timelineEvents)`), because the Timeline domain’s fixtures were not included in this isolation sweep’s import list. This is a real, present-day gap in the isolation guarantee, not a field-data blocker — see the technical report §3/§6.

## 4. Lint / type checks

No lint or type-check command exists in this repository.

- `package.json` scripts: `dev`, `build`, `preview`, `test` only — no `lint` or `typecheck` script.
- No `.eslintrc*` file anywhere in the repository root or `src/`.
- No `tsconfig.json`; all source files are plain `.js` / `.jsx`.

**Result: N/A — not configured.** This is reported as a gap, not assumed to be silently passing.

## 5. Security / static-analysis checks

```
$ npm audit
# npm audit report

esbuild <=0.24.2
Severity: moderate
esbuild enables any website to send any requests to the development server and read the response - https://github.com/advisories/GHSA-67mh-4wv8-2f99
fix available via `npm audit fix --force`
Will install vite@8.3.0, which is a breaking change
node_modules/esbuild
  vite <=6.4.2
  Depends on vulnerable versions of esbuild
  node_modules/vite

2 vulnerabilities (1 moderate, 1 high)
```

```
$ npm audit --json | jq .metadata.vulnerabilities
{ "info": 0, "low": 0, "moderate": 1, "high": 1, "critical": 0, "total": 2 }
```

**Result:** 2 known vulnerabilities, both in **development-only** tooling ( `esbuild` / `vite` ), not in runtime production dependencies ( `react`, `react-dom` — 2 total runtime deps). No secrets, credentials, or hard-coded tokens were found during this audit's manual review of `src/`. No dedicated static-analysis tool (e.g. CodeQL, Semgrep) is configured in this repository; `npm audit` is the only security tooling actually runnable here.

## 6. Existing CI-equivalent commands

There is **no CI configuration** ( `.github/workflows/*` absent). The commands executed in this audit ( `npm install`, `npm run build`, `npm test`, `npm audit` ) represent the full set of automatable checks this repository currently defines via `package.json`. They were run here for the first time in this audit environment; no historical CI run record exists to compare against.

## 7. Summary

| Check             | Command                    | Result                                                                             |
|-------------------|----------------------------|------------------------------------------------------------------------------------|
| Install           | <code>npm install</code>   | PASS (63 packages, 2 known vulns in dev deps)                                      |
| Build             | <code>npm run build</code> | PASS (1 non-fatal bundle-size warning)                                             |
| Unit/domain tests | <code>npm test</code>      | PASS — 395 assertions across 4 counted suites + 1 full isolation sweep, 0 failures |
| Lint              | <i>(none configured)</i>   | N/A                                                                                |
| Type check        | <i>(none configured)</i>   | N/A                                                                                |
| Security scan     | <code>npm audit</code>     | 2 vulnerabilities (1 moderate, 1 high), dev-dependency only                        |
| CI                | <i>(none configured)</i>   | N/A — no <code>.github/workflows</code>                                            |